CS-30003 · Coding Assignment 1 · Project Proposal

Team Sohan Mandal (2405912, CSE-14) · Sinjan Mishra (2405910, CSE-14) · Sohini Pandit (2405913, CSE-14) · Shaili Seth (2405901, CSE-14)
Repository https://github.com/sohan1611/rdt-udp (access shared with instructor)
Project P3 · Reliable Data Transfer over UDP
Path A (catalogue)

1. WHAT WE ARE BUILDING

A file-transfer system built directly on UDP datagram sockets, with the reliability, ordering and integrity guarantees implemented by us at the transport layer. One application interface sits above three interchangeable ARQ transports — Stop-and-Wait, Go-Back-N and Selective Repeat — chosen by a command-line flag with no change to application code. Between the endpoints we run our own channel emulator as a **separate middlebox process**, so the protocol contains no test-only code paths and genuinely does not know it is being measured. Below our layer is only UDP's unreliable datagram service; above it the application sees an ordered, hash-verified file.

2. CORE DELIVERABLES

- 20-byte hand-packed header via `ByteBuffer` (big-endian, hence already network byte order), and the 16-bit one's-complement Internet checksum of RFC 1071 written by hand.
- Three swappable transports behind one interface: alternating-bit Stop-and-Wait; Go-Back-N with a sliding window, cumulative ACKs and full-window retransmit; Selective Repeat with per-packet timers, selective ACKs, receiver-side buffering of out-of-order packets and in-order delivery.
- Retransmission timers on a **single-threaded timer heap** driven by one socket loop — no timer thread, therefore no race between a timeout firing and an ACK arriving; both are events on the same loop.
- Adaptive RTO via Jacobson/Karels (SRTT, RTTVAR) with Karn's algorithm and exponential backoff.
- Channel emulator written by us as a standalone process: configurable loss, duplication, reordering, corruption and delay with jitter, seeded with `java.util.Random`, whose algorithm is fixed by the Java specification so a seed reproduces byte-identically on any machine. Every packet consumes a fixed number of draws, so changing the loss rate does not shift the delay sequence and two cells of a sweep differ only in the variable under test.
- A **JSONL trace of every impairment decision**, giving per-packet ground truth on whether a packet was genuinely lost — which lets us classify a retransmission as spurious rather than guess.
- SHA-256 comparison of source against received file after every transfer; per-run statistics (goodput, wire throughput, retransmissions, timeouts, duplicate ACKs) to CSV; a harness regenerating every figure from those CSVs.

3. THE CLAIM WE WILL TEST

Claim. Under loss, Selective Repeat's goodput degrades approximately as $(1-p)$ while Go-Back-N's degrades approximately as $(1-p)^W$, so the gap widens with both loss rate and window size, while Stop-and-Wait stays bounded by MSS/RTT regardless of either. This is falsifiable because it predicts a functional *form*, not merely an ordering: if the measured Go-Back-N curve does not track $(1-p)^W$, we report that and explain why.

Experiment. *Independent variables* — loss rate (0, 0.1, 0.5, 1, 2, 5, 10, 20%), window size (1–128), reordering probability (0–20%), RTO policy (fixed at 1.2/1.5/2/3×RTT versus adaptive). *Controls* — same 8 MiB file, 1400-byte payload, emulated RTT and jitter, and the same seed set across all three protocols; ten seeds per cell. *Metrics* — goodput and wire throughput reported separately so retransmission overhead is visible rather than hidden; retransmission count; and, using the emulator trace as ground truth, the fraction of retransmissions that were *spurious* because the original had in fact arrived. *Presentation* — four plots, each with mean and 95% confidence intervals over seeds and a written explanation of the curve's shape. *Validation* — we re-run the loss sweep under Linux `tc netem` and show our emulator reproduces the same curve, and we publish the measured harness capacity ceiling (64,000 packets/second on our test host) so every data point is visibly below it.

4. STACK AND JUSTIFICATION

Java 21 (`java.net`, `java.nio`) for the protocols and emulator; Python 3 with pandas and matplotlib for analysis only; Linux via WSL2. Java because the viva is assessed individually and most of the group is strongest in Java, so it maximises how well each of us can explain our own code, and because `ByteBuffer` is big-endian by default, which is already network byte order. We benchmarked the brief's performance warning rather than assuming it: a textbook RFC 1071 checksum runs at roughly 14,000 packets/second in Python but 1.68 million in Java, against an operating point near 4,500 — so in Java we can write the checksum exactly as the RFC specifies instead of in an optimised form. Mixing is permitted, and keeping analysis in Python separates the system from the measurement of it.

5. WORK SPLIT

COMPONENT	OWNER
Wire format, checksum, metadata handshake, Stop-and-Wait, sender and receiver CLIs	**Shaili Seth**
Timer heap, Jacobson/Karels and Karn RTO estimation, Go-Back-N	**Sinjan Mishra**
Selective Repeat: receiver buffer, selective ACKs, sequence-space wraparound correctness	**Sohini Pandit**
Channel emulator, capacity calibration, experiment harness, statistics, plots	**Sohan Mandal**
Own-module tests; pull-request review including a spoken explanation of the change; `AI-USE.md`; final report	**All**

6. RISKS

- **Selective Repeat overruns its week.** Per-packet timers plus a receiver buffer is the hardest single piece of the project. *Fallback:* ship it with one timer on the oldest unacknowledged packet — still correct, still ahead of Go-Back-N — documented as a deliberate simplification, with per-packet timers moved to stretch. We take this decision on 25 September, not in October.
- **JVM timing jitter contaminating the measurements.** JIT compilation makes the first few thousand packets of a run slower than the rest, and garbage-collection pauses look exactly like network delay in a latency measurement. *Fallback:* discard a warm-up transfer per JVM, pin the heap so it never resizes, keep the hot path free of allocation, and log GC to confirm no collection occurred during any timed run.

7. WEEKLY PLAN

WEEK	TARGET	OWNER
1 · Sep 1–7	Repository and `AI-USE.md`; packet format and checksum with unit tests; emulator v0 (loss, seeded); harness capacity calibration	**Shaili, Sohan**
2 · Sep 8–14	Emulator v1 (duplication, reordering, corruption, delay with jitter, JSONL trace); timer heap; Stop-and-Wait end to end with SHA-256 match	**Sohan, Sinjan**
3 · Sep 15–21	Go-Back-N: send window, cumulative ACKs, timeout retransmit; per-run statistics to CSV; first plot	**Sinjan, Sohan**
4 · Sep 22–28	Selective Repeat: per-packet timers, receiver buffer, in-order delivery; adaptive RTO across all three. Core complete	**Sohini, Sinjan**
5 · Sep 29–Oct 5	Experiments 1–3 at full seed count; methodology write-up	**Sohan, Sohini**
6 · Oct 6–12	Experiment 4 with spurious-retransmission labelling; `tc netem` cross-validation; cross-teaching so each member can trace a packet through code they did not write	**Sohan, All**
7 · Oct 13–19	Stretch attempt (SACK blocks); full report draft reviewed by all four	**Sohini, Shaili**
8 · Oct 20–26	Code freeze; final analysis write-up; repository cleanup; viva preparation and live-modification drills	**All**